

14

Text classification

This chapter covers

- An introduction to the field of natural language processing (NLP)
- Preprocessing text input into numeric input
- Building simple text classification models

This chapter will lay the foundation for working with text input that we will build on in the next two chapters of this book. By the end of this chapter, you will be able to build a simple text classifier in a number of different ways. This will set the stage for building more complicated models, like the Transformer, in the next chapter.

14.1 A brief history of natural language processing

In computer science, we refer to human languages, like English or Mandarin, as “natural” languages to distinguish them from languages that were designed for machines, like LISP, Assembly, and XML. Every machine language was designed: its starting point was an engineer writing down a set of formal rules to describe what statements you can make and what they mean. The rules came first, and people only started using the language once the rule set was complete. With human language,

it's the reverse: usage comes first, and rules arise later. Natural language was shaped by an evolutionary process, much like biological organisms—that's what makes it "natural." Its "rules," like the grammar of English, were formalized after the fact and are often ignored or broken by its users. As a result, while machine-readable language is highly structured and rigorous, natural language is messy—ambiguous, chaotic, sprawling, and constantly in flux.

Computer scientists have long fixated on the potential of systems that can ingest or produce natural language. Language, particularly written text, underpins most of our communications and cultural production. Centuries of human knowledge are stored via text; the internet is mostly text, and even our thoughts are based on language! The practice of using computers to interpret and manipulate language is called natural language processing, or NLP for short. It was first proposed as a field of study immediately following World War II, where some thought we could view understanding language as a form of "code cracking," where natural language is the "code" used to transmit information.

In the early days of the field, many people naively thought that you could write down the "rule set of English," much like one can write down the rule set of LISP. In the early 1950s, researchers at IBM and Georgetown demonstrated a system that could translate Russian into English. The system used a grammar with six hardcoded rules and a lookup table with a couple of hundred elements (words and suffixes) to translate 60 handpicked Russian sentences accurately. The goal was to drum up excitement and funding for machine translation, and in that sense, it was a huge success. Despite the limited nature of the demo, the authors claimed that within five years, translation would be a solved problem. Funding poured in for the better part of a decade. However, generalizing such a system proved to be maddeningly difficult. Words change their meaning dramatically depending on context. Any grammar rules needed countless exceptions. Developing a program that could shine on a few handpicked examples was simple enough, but building a robust system that could compete with human translators was another matter. An influential US report a decade later picked apart the lack of progress, and funding dried up.

Despite these setbacks and repeated swings from excitement to disillusionment, handcrafted rules held out as the dominant approach well into the 1990s. The problems were obvious, but there was simply no viable alternative to writing down symbolic rules to describe grammar. However, as faster computers and greater quantities of data became available in the late 1980s, research began to head in a new direction. When you find yourself building systems that are big piles of ad hoc rules, as a clever engineer, you're likely to start asking, "Could I use a corpus of data to automate the process of finding these rules? Could I search for the rules within some rule space, instead of having to come up with them myself?" And just like that, you've graduated to doing machine learning.

In the late 1980s, we started seeing machine learning approaches to natural language processing. The earliest ones were based on decision trees—the intent was literally to

automate the development of the kind of if/then/else rules of hardcoded language systems. Then, statistical approaches started gaining speed, starting with logistic regression. Over time, learned parametric models took over, and linguistics came to be seen by some as a hindrance when baked directly into a model. Frederick Jelinek, an early speech recognition researcher, joked in the 1990s, “Every time I fire a linguist, the performance of the speech recognizer goes up.”

Much as computer vision is pattern recognition applied to pixels, the modern field of NLP is all about pattern recognition applied to words in text. There’s no shortage of practical applications:

- Given the text of an email, what is the probability that it is spam? (*text classification*)
- Given an English sentence, what is the most likely Russian translation? (*translation*)
- Given an incomplete sentence, what word will likely come next? (*language modeling*)

The text-processing models you will train in this book won’t possess a human-like understanding of language; rather, they simply look for statistical regularities in their input data, which turns out to be sufficient to perform well on a wide array of real-world tasks.

In the last decade, NLP researchers and practitioners have discovered just how shockingly effective it can be to learn the answer to narrow statistical questions about text. In the 2010s, researchers began applying LSTM models to text, dramatically increasing the number of parameters in NLP models and the compute resources required to train them. The results were encouraging—LSTMs could generalize to unseen examples with far greater accuracy than previous approaches, but they eventually hit limits. LSTMs struggled to track dependencies in long chains of text with many sentences and paragraphs, and compared to computer vision models, they were slow and unwieldy to train.

Toward the end of the 2010s, researchers at Google discovered a new architecture called the Transformer that solved many scalability issues plaguing LSTMs. As long as you increased the size of a model and its training data together, Transformers appeared to perform more and more accurately. Better yet, the computations needed for training a Transformer could be effectively parallelized, even for long sequences. If you doubled the number of machines doing training, you could roughly halve the time you need to wait for a result.

The discovery of the Transformer architecture, along with ever-faster GPUs and CPUs, has led to a dramatic explosion of investment and interest in NLP models over the past few years. Chat systems like ChatGPT have captivated public attention with their ability to produce fluent and natural text on seemingly arbitrary topics and questions. The raw text used to train these models is a significant portion of all written language available on the internet, and the compute to train individual models can cost tens of millions of dollars. Some hype is worth cutting down to size—these are pattern

recognition machines. Despite our persistent human tendency to find intelligence in “things that talk,” these models copy and synthesize training data in a way that is wholly distinct (and much less efficient!) than human intelligence. However, it is also fair to say that the emergence of complex behaviors from incredibly simple “guess the missing word” training setups has been one of the most shocking empirical results in the last decade of machine learning.

In the following three chapters, we will look at a range of techniques for machine learning with text data. We will skip discussion of the hardcoded linguistic features that prevailed until the 1990s, but we will look at everything from running logistic regressions for classifying text to training LSTMs for machine translation. We will closely examine the Transformer model and discuss what makes it so scalable and effective at generalizing in the text domain. Let’s dig in.

14.2 Preparing text data

Let’s consider an English sentence:

```
The quick brown fox jumped over the lazy dog.
```

There is an obvious blocker before we can start applying any of the deep learning techniques of previous chapters—our input is not numeric! Before beginning any modeling, we need to translate the written word into tensors of numbers. Unlike images, which have a relatively natural numeric representation, you could build a numeric representation of text in several ways.

A simple approach would be to borrow from standard text file formats for text and use something like an ASCII encoding. We could chop the input into a sequence of characters and assign each a unique index. Another intuitive approach would be building a representation based on words, first breaking sentences apart on all spaces and punctuation and then mapping each word to a unique numeric representation.

These are both good approaches to try, and in general, all text preprocessing will include a *splitting* step, where text is split into small individual units, called *tokens*. A powerful tool for splitting text is regular expressions, which can flexibly match patterns of characters in text.

Let’s look at how to use a regular expression to split a string into a sequence of characters. The most basic regex we can apply is `"."`, which matches any character in the input text:

```
import regex as re

def split_chars(text):
    return re.findall(r".", text)
```

We can apply the function to our example input string:

```
>>> chars = split_chars("The quick brown fox jumped over the lazy dog.")
>>> chars[:12]
["T", "h", "e", " ", "q", "u", "i", "c", "k", " ", "b", "r"]
```

Regex can easily be applied to split our text into words instead. The `"[\w]+"` regular expression will grab consecutive non-whitespace characters, and the `"[.,!?!;]"` can match the punctuation marks between the brackets. We can combine the two to achieve a regular expression that splits each word and punctuation mark into a token:

```
def split_words(text):
    return re.findall(r"[\w]+|[.,!?!;]", text)
```

Here's what it does to a test sentence:

```
>>> split_words("The quick brown fox jumped over the dog.")
["The", "quick", "brown", "fox", "jumped", "over", "the", "dog", "."]
```

Splitting takes us from a single string to a token sequence, but we still need to transform our string tokens into numeric inputs. By far the most common approach is to map each token to a unique integer index, often called *indexing* our input. This is a flexible and reversible representation of our tokenized input that can work with a wide range of modeling approaches. Later on, we can decide how to map from token indices into a latent space ingested by the model.

For character tokens, we could use ASCII lookups to index each token—for example, `ord('A') → 65` and `ord('z') → 122`. However, this can scale poorly when you start to consider other languages—there are over a million characters in the Unicode specification! A more robust technique is to build a mapping from specific tokens in our training data to indices that occur in the data we care about, which in NLP is called a *vocabulary*. This has the nice property of working for word-level tokens as easily as for character-level tokens.

Let's take a look at how we might use a vocabulary to transform text. We will build a simple Python dictionary that maps tokens to indices, split our input into tokens, and finally index our tokens:

```
vocabulary = {
    "[UNK]": 0,
    "the": 1,
    "quick": 2,
    "brown": 3,
    "fox": 4,
    "jumped": 5,
```

```

    "over": 6,
    "dog": 7,
    ".": 8,
}
words = split_words("The quick brown fox jumped over the lazy dog.")
indices = [vocabulary.get(word, 0) for word in words]

```

This outputs the following:

```
[0, 2, 3, 4, 5, 6, 1, 0, 7, 8]
```

We introduce a special token called "[UNK]" to our vocabulary, which represents a token that is unknown to the vocabulary. This way, we can index all input we come across, even if some terms only occur in our test data. In the previous example "lazy" maps to the "[UNK]" index 0, as it was not included in our vocabulary.

With these simple text transformations, we are well on our way to building a text pre-processing pipeline. However, there is one more common type of text manipulation we should consider—standardization.

Consider these two sentences:

- “sunset came. i was staring at the Mexico sky. Isn't nature splendid??”
- “Sunset came; I stared at the México sky. Isn’t nature splendid?”

They are very similar—in fact, they are almost identical. Yet, if you were to convert them to indices as previously described, you would end up with very different representations because “i” and “I” are two distinct characters, “Mexico” and “México” are two distinct words, “isn't” isn’t “isn’t,” and so on. A machine learning model doesn’t know a priori that “i” and “I” are the same letter, that “é” is an “e” with an accent, or that “staring” and “stared” are two forms of the same verb. *Standardizing* text is a basic form of feature engineering that aims to erase encoding differences that you don’t want your model to have to deal with. It’s not exclusive to machine learning, either—you’d have to do the same thing if you were building a search engine.

One simple and widespread standardization scheme is to convert to lowercase and remove punctuation characters. Our two sentences would become

- “sunset came i was staring at the mexico sky isnt nature splendid”
- “sunset came i stared at the méxico sky isnt nature splendid”

Much closer already. We could get even closer if we removed accent marks on all characters.

There’s a lot you can do with standardization, and it used to be one of the most critical areas to improve model performance. For many decades in NLP, it was common practice to use regular expressions to attempt to map words to a common root (e.g. “tired” → “tire” and “trophies” → “trophy”), called *stemming* or *lemmatization*. But

as models have grown more expressive, this type of standardization tends to do more harm than good. The tense and plurality of a word are necessary signals to its meaning. For the larger models used today, most standardization is as light as possible—for example, converting all inputs to a standard character encoding before further processing.

With standardization, we have now seen three distinct stages for preprocessing text (figure 14.1):

- 1 *Standardization*—Where we normalize input with basic text-to-text transformations
- 2 *Splitting*—Where we split our text into sequences of *tokens*
- 3 *Indexing*—Where we map our tokens to indices using a *vocabulary*

People often refer to the entire process as *tokenization*, and to an object that maps text to sequence of token indices as a *tokenizer*. Let's try building a few.

14.2.1 Character and word tokenization

To start, let's build a character-level tokenizer that maps each character in an input string to an integer. To keep things simple, we will use only one standardization step—we lowercase all input.

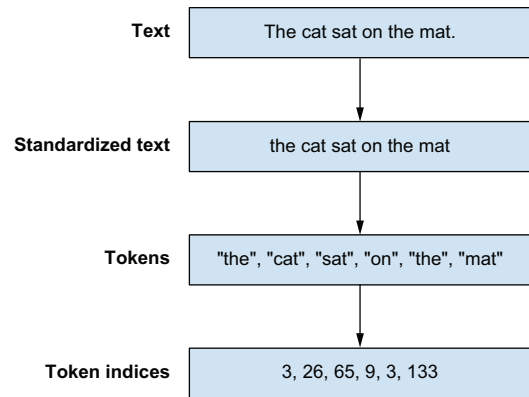


Figure 14.1 The text preprocessing pipeline

Listing 14.1 A basic character-level tokenizer

```

class CharTokenizer:
    def __init__(self, vocabulary):
        self.vocabulary = vocabulary
        self.unk_id = vocabulary["[UNK]"]

    def standardize(self, inputs):
        return inputs.lower()

    def split(self, inputs):
        return re.findall(r".", inputs)

    def index(self, tokens):
        return [self.vocabulary.get(t, self.unk_id) for t in tokens]

    def __call__(self, inputs):
        inputs = self.standardize(inputs)
        tokens = self.split(inputs)
  
```

```
indices = self.index(tokens)
return indices
```

Pretty simple. Before using this, we also need to build a function that computes a vocabulary of tokens based on some input text. Rather than simply mapping all characters to a unique index, let's give ourselves the ability to limit our vocabulary size to only the most common tokens in our input data. When we get into the modeling side of things, limiting the vocabulary size will be an important way to limit the number of parameters in a model.

Listing 14.2 Computing a character-level vocabulary

```
import collections

def compute_char_vocabulary(inputs, max_size):
    char_counts = collections.Counter()
    for x in inputs:
        x = x.lower()
        tokens = re.findall(r".", x)
        char_counts.update(tokens)
    vocabulary = ["[UNK]"]
    most_common = char_counts.most_common(max_size - len(vocabulary))
    for token, count in most_common:
        vocabulary.append(token)
    return dict((token, i) for i, token in enumerate(vocabulary))
```

We can now do the same for a word-level tokenizer. We can use the same code as our character-level tokenizer with a different splitting step.

Listing 14.3 A basic word-level tokenizer

```
class WordTokenizer:
    def __init__(self, vocabulary):
        self.vocabulary = vocabulary
        self.unk_id = vocabulary["[UNK]"]

    def standardize(self, inputs):
        return inputs.lower()

    def split(self, inputs):
        return re.findall(r"[\w]+|[.,!?:;]", inputs)

    def index(self, tokens):
        return [self.vocabulary.get(t, self.unk_id) for t in tokens]

    def __call__(self, inputs):
        inputs = self.standardize(inputs)
        tokens = self.split(inputs)
```



```
indices = self.index(tokens)
return indices
```

We can also substitute this new split rule into our vocabulary function.

Listing 14.4 Computing a word-level vocabulary

```
def compute_word_vocabulary(inputs, max_size):
    word_counts = collections.Counter()
    for x in inputs:
        x = x.lower()
        tokens = re.findall(r"[\w]+|[.,!?;]", x)
        word_counts.update(tokens)
    vocabulary = ["[UNK]"]
    most_common = word_counts.most_common(max_size - len(vocabulary))
    for token, count in most_common:
        vocabulary.append(token)
    return dict((token, i) for i, token in enumerate(vocabulary))
```

Let's try out our tokenizers on some real-world input—the full text of *Moby Dick* by Herman Melville. We will first build a vocabulary for both tokenizers and then use it to tokenize some text:

```
import keras

filename = keras.utils.get_file(
    origin="https://www.gutenberg.org/files/2701/old/moby10b.txt",
)
moby_dick = list(open(filename, "r"))

vocabulary = compute_char_vocabulary(moby_dick, max_size=100)
char_tokenizer = CharTokenizer(vocabulary)
```

Let's inspect what our character-level tokenizer has computed:

```
>>> print("Vocabulary length:", len(vocabulary))
Vocabulary length: 64
>>> print("Vocabulary start:", list(vocabulary.keys())[:10])
Vocabulary start: ["[UNK]", " ", "e", "t", "a", "o", "n", "i", "s", "h"]
>>> print("Vocabulary end:", list(vocabulary.keys())[-10:])
Vocabulary end: ["@", "$", "%", "#", "=", "~", "&", "+", "<", ">"]>>>
print("Line length:", len(char_tokenizer(
...     "Call me Ishmael. Some years ago--never mind how long precisely."
... )))
Line length: 63
```

Now what about the word-level tokenizer?

```
vocabulary = compute_word_vocabulary(moby_dick, max_size=2_000)
word_tokenizer = WordTokenizer(vocabulary)
```

We can print out the same data for our word-level tokenizer:

```
>>> print("Vocabulary length:", len(vocabulary))
Vocabulary length: 2000
>>> print("Vocabulary start:", list(vocabulary.keys())[:5])
Vocabulary start: ["[UNK]", ",", "the", ".", "of"]
>>> print("Vocabulary end:", list(vocabulary.keys())[-5:])
Vocabulary end: ["tambourine", "subtle", "perseus", "elevated", "repose"]
>>> print("Line length:", len(word_tokenizer(
...     "Call me Ishmael. Some years ago--never mind how long precisely."
... )))
Line length: 13
```

We can already see some of the strengths and weaknesses of both tokenization techniques. A character-level tokenizer needs only 64 vocabulary terms to cover the entire book but will encode each input as a very long sequence. A word-level tokenizer quickly fills a 2,000-term vocabulary (you would need a dictionary with 17,000 terms to index every word in the book!), but the outputs of the word-level tokenizer are much shorter.

As machine learning practitioners have scaled models up with more and more data and parameters, the downsides of both word and character tokenization have become apparent. The “compression” offered by word-level tokenization turns out to be very important—it allows feeding longer sequences into a model. However, if you attempt to build a word-level vocabulary for a large dataset (today, you might see a dataset with trillions of words), you would have an unworkably large vocabulary with hundreds of millions of terms. If you aggressively restrict your word-level vocabulary size, you will encode a lot of text to the “[UNK]” token, throwing out valuable information.

These issues have led to the rise in popularity of a third type of tokenization, called *subword tokenization*, which attempts to bridge the gap between word- and character-level approaches.

14.2.2 Subword tokenization

Subword tokenization aims to combine the best of both character- and word-level encoding techniques. We want the `WordTokenizer`’s ability to produce concise output and the `CharTokenizer`’s ability to encode a wide range of inputs with a small vocabulary.

We can think of the search for the ideal tokenizer as the hunt for an ideal compression of the input data. Reducing token length compresses the overall length of our examples. A small vocabulary reduces the number of bytes we would need to represent each token. If we achieve both, we will be able to feed short, information-rich sequences to our deep learning model.

This analogy between compression and tokenization was not always obvious, but it turns out to be powerful. One of the most practically effective tricks found in the last decade of NLP research was repurposing a 1990s algorithm for lossless compression called *byte-pair encoding*¹ for tokenization. It is used by ChatGPT and many other models to this day. In this section, we will build a tokenizer that uses the byte-pair encoding algorithm.

The idea with byte-pair encoding is to start with a basic vocabulary of characters and progressively “merge” common pairings into longer and longer sequences of characters. Let’s say we start with the following input text:

```
data = [
    "the quick brown fox",
    "the slow brown fox",
    "the quick brown foxhound",
]
```

Like the `WordTokenizer`, we will start by computing word counts for all the words in the text. As we create our dictionary of word counts, we will split all our text into characters and join the characters with a space. This will make it easier to consider pairs of characters in our next step.

Listing 14.5 Initializing state for the byte-pair encoding algorithm

```
def count_and_split_words(data):
    counts = collections.Counter()
    for line in data:
        line = line.lower()
        for word in re.findall(r"[\w]+|[.,!?:;]", line):
            chars = re.findall(r".", word)
            split_word = " ".join(chars)
            counts[split_word] += 1
    return dict(counts)

counts = count_and_split_words(data)
```

Let’s try this out on our data:

```
>>> counts
{"t h e": 3,
 "q u i c k": 2,
 "b r o w n": 3,
 "f o x": 2,
 "s l o w": 1,
 "f o x h o u n d": 1}
```

¹ Phillip Gage, “A New Algorithm for Data Compression,” *The C Users Journal Archive* (1994), <https://dl.acm.org/doi/10.5555/177910.177914>.

To apply byte-pair encoding to our split word counts, we will find two characters and merge them into a new symbol. We consider all pairs of characters in all words and only merge the most common one we find. In the previous example, the most common character pair is ("o", "w"), in both the word "brown" (which occurs three times in our data) and "slow" (which occurs once). We combine this pair into a new symbol "ow" and merge all occurrences of "o w".

Then we continue, counting pairs and merging pairs, except now "ow" will be a single unit that could merge with, say, "l" to form "low". By progressively merging the most frequent symbol pair, we build up a vocabulary of larger and larger subwords.

Let's try this out on our toy dataset.

Listing 14.6 Running a few steps of byte-pair merging

```
def count_pairs(counts):
    pairs = collections.Counter()
    for word, freq in counts.items():
        symbols = word.split()
        for pair in zip(symbols[:-1], symbols[1:]):
            pairs[pair] += freq
    return pairs

def merge_pair(counts, first, second):
    split = re.compile(f"(?<!\S){first} {second}(?!\\S)")
    merged = f"{first}{second}"
    return {split.sub(merged, word): count for word, count in counts.items()}

for i in range(10):
    pairs = count_pairs(counts)
    first, second = max(pairs, key=pairs.get)
    counts = merge_pair(counts, first, second)
    print(list(counts.keys()))
```

Matches an unmerged pair

Replaces all occurrences with a merged version

Here's what we get:

```
["t h e", "q u i c k", "b r o w n", "f o x", "s l o w", "f o x h o u n d"]
["th e", "q u i c k", "b r o w n", "f o x", "s l o w", "f o x h o u n d"]
["the", "q u i c k", "b r o w n", "f o x", "s l o w", "f o x h o u n d"]
["the", "q u i c k", "br ow n", "f o x", "s l o w", "f o x h o u n d"]
["the", "q u i c k", "brown", "f o x", "s l o w", "f o x h o u n d"]
["the", "q u i c k", "brown", "fo x", "s l o w", "fo x h o u n d"]
["the", "q u i c k", "brown", "fox", "s l o w", "fox h o u n d"]
["the", "qu i c k", "brown", "fox", "s l o w", "fox h o u n d"]
["the", "qui c k", "brown", "fox", "s l o w", "fox h o u n d"]
```

We can see how common words are merged entirely, whereas less common words are only partially merged.

We can now extend this to a full function for computing a byte-pair encoding vocabulary. We start our vocabulary with all characters found in the input text, and we will progressively add merged symbols (larger and larger subwords) to our vocabulary until it reaches our desired length. We also keep a separate dictionary of our merge rules, including a rank order in which we applied them. Next, we will see how to use these merge rules to tokenize new input text.

Listing 14.7 Computing a byte-pair encoding vocabulary

```
def compute_sub_word_vocabulary(dataset, vocab_size):
    counts = count_and_split_words(dataset)

    char_counts = collections.Counter()
    for word in counts:
        for char in word.split():
            char_counts[char] += counts[word]
    most_common = char_counts.most_common()
    vocab = ["[
        <
        <!\S)\S+ \S+(?!\\S)", word, overlapped=True)
        if not pairs:
            break
        best = min(pairs, key=lambda pair: self.merges.get(pair, 1e9))
        if best not in self.merges:
            break
        first, second = best.split()
        split = re.compile(f"(?!\\S){first} {second}(?!\\S)")
        merged = f"{first}{second}"
        word = split.sub(merged, word)
    return word

def split(self, inputs):
    tokens = []
    for word in re.findall(r"[
        word=self.bpe_merge(word)
        tokens.extend(word.split())
    return tokens

def index(self, tokens):
    return [self.vocabulary.get(t, self.unk_id) for t in tokens]

def __call__(self, inputs):
    inputs = self.standardize(inputs)
    tokens = self.split(inputs)
    indices = self.index(tokens)
    return indices
```

Matches all symbol pairs in the text

We apply merge rules in "rank" order. More frequent pairs are merged first.

Split words

Joins all characters with a space

Applies byte-pair encoding merge rules

Let's try out our tokenizer on the full text of *Moby Dick*:

```
vocabulary, merges = compute_sub_word_vocabulary(moby_dick, 2_000)
sub_word_tokenizer = SubWordTokenizer(vocabulary, merges)
```

We can take a look at our vocabulary and try a test sentence on our tokenizer, as we did with `WordTokenizer` and `CharTokenizer`:

```
>>> print("Vocabulary length:", len(vocabulary))
Vocabulary length: 2000
>>> print("Vocabulary start:", list(vocabulary.keys())[:10])
Vocabulary start: ["[UNK]", "e", "t", "a", "o", "n", "i", "s", "h", "r"]
>>> print("Vocabulary end:", list(vocabulary.keys())[-7:])
Vocabulary end: ["bright", "pilot", "sco", "ben", "dem", "gale", "ilo"]
>>> print("Line length:", len(sub_word_tokenizer(
...     "Call me Ishmael. Some years ago--never mind how long precisely."
... )))
Line length: 16
```

The `SubWordTokenizer` has a slightly longer length for our test sentence than the `WordTokenizer` (16 versus 13 tokens), but unlike the `WordTokenizer`, it can tokenize every word in *Moby Dick* without using the "[UNK]" token. The vocabulary contains every character in our source text, so the worst-case performance will be tokenizing a word into individual characters. We have achieved a short *average* token length while handling rare words with a small vocabulary. This is the advantage of subword tokenizers.

You might notice that running this code is noticeably slower than the word and character tokenizers; it takes about a minute on our reference hardware. Learning merge rules is much more complex than simply counting the words in an input dataset. While this is a downside to subword tokenization, it is rarely an important concern in practice. You only need to learn a vocabulary once per model, and the cost of learning a subword vocabulary is generally negligible compared to model training.

We have now seen three separate approaches for tokenizing input. Now that we can translate from text to numeric input, we can move on to training a model.

One final note on tokenization—while it is quite important to understand how tokenizers work, it is rarely the case that you will need to build one yourself. Keras comes with utilities for tokenizing text input, as do most deep learning frameworks. For the rest of the chapter, we will make use of the built-in functionality in Keras for tokenization.

Which tokenization technique should I use?

When approaching a new text-modeling problem, one of the first questions you will need to answer is how to tokenize your input. As we will see at the end of this chapter, the question is trivial for a given pretrained model. You have to preserve the exact

tokenization used during pretraining or throw out the useful representations of input tokens contained in the model weights.

If you are building a model from scratch, you can tailor your tokenization to the problem at hand. In general, the compression offered by word and subword tokenizers is too important to pass up. The shorter the length of your inputs on average, the better a model will be able to track long-range dependencies in the text, improving its overall performance. This has made subword the dominantly popular choice for modern language models. They can handle rare or misspelled words without inflating token length for common inputs.

However, there is no one-size-fits-all approach. Some problems in NLP, such as spelling correction, might benefit from low-level character tokenization of input text. On the other hand, a word-level approach is both simple to work with and easily understandable—each model input corresponds to a word a human would read. This would make ranking tokens by importance to a prediction easy to interpret.

We will use all three types of tokenizers throughout the text chapters of this book.

14.3 Sets vs. sequences

How a machine learning model should represent individual tokens is a relatively uncontroversial question: they're categorical features (values from a predefined set), and we know how to handle those. They should be encoded as dimensions in a feature space or as category vectors (token vectors in this case). A much more problematic question, however, is how to encode the ordering of tokens in text.

The problem of order in natural language is an interesting one: unlike the steps of a timeseries, words in a sentence don't have a natural, canonical order. Different languages order similar words in very different ways. For instance, the sentence structure of English is quite different from that of Japanese. Even within a given language, you can typically say the same thing in different ways by reshuffling the words a bit. If you were to fully randomize the words in a short sentence, you could still sometimes figure out what it was saying—though, in many cases, significant ambiguity would arise. Order is clearly important, but its relationship to meaning isn't straightforward.

How to represent word order is the pivotal question from which different kinds of NLP architectures spring. The simplest thing you could do is discard order and treat text as an unordered set of words—this gives you bag-of-words models. You could also decide that words should be processed strictly in the order in which they appear, one at a time, like steps in a timeseries—you could then use the recurrent models from the previous chapter. Finally, a hybrid approach is also possible: the Transformer architecture is technically order-agnostic, yet it injects word-position information into the representations it processes, which enables it to simultaneously look at different parts of a sentence (unlike RNNs) while still being order-aware. Because they take into account word order, both RNNs and Transformers are called *sequence models*.

Historically, most early applications of machine learning to NLP just involved bag-of-words models that discarded sequence data. Interest in sequence models only started rising in 2015, with the rebirth of RNNs. Today, both approaches remain relevant. Let's see how they work and when to use which.

We will demonstrate each approach on a well-known text classification benchmark: the IMDb movie review sentiment-classification dataset. In chapters 4 and 5, you worked with a prevectorized version of the IMDb dataset; now let's process the raw IMDb text data, just like you would do when approaching a new text-classification problem in the real world.

14.3.1 Loading the IMDb classification dataset

To begin, let's download and extract our dataset.

Listing 14.8 Downloading the IMDb movie review dataset

```
import os, pathlib, shutil, random

zip_path = keras.utils.get_file(
    origin="https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz",
    fname="imdb",
    extract=True,
)

imdb_extract_dir = pathlib.Path(zip_path) / "aclImdb"
```

Let's list out our directory structure:

```
>>> for path in imdb_extract_dir.glob("*/*"):
...     if path.is_dir():
...         print(path)
~/keras/datasets/aclImdb/train/pos
~/keras/datasets/aclImdb/train/unsup
~/keras/datasets/aclImdb/train/neg
~/keras/datasets/aclImdb/test/pos
~/keras/datasets/aclImdb/test/neg
```

We can see both a train and test set with positive and negative examples. Movie reviews with a low user rating on the IMDb site were sorted into the `neg/` directory and those with a high rating into the `pos/` directory. We can also see an `unsup/` directory, which is short for unsupervised. These are reviews deliberately left unlabeled by the dataset creator; they could be negative or positive reviews.

Let's look at the content of a few of these text files. Whether you're working with text or image data, remember to inspect what your data looks like before you dive into modeling. It will ground your intuition about what your model is actually doing.

Listing 14.9 Previewing a single IMDb review

```
>>> print(open(imdb_extract_dir / "train" / "pos" / "4077_10.txt", "r").read())
I first saw this back in the early 90s on UK TV, i did like it then but i missed
the chance to tape it, many years passed but the film always stuck with me and i
lost hope of seeing it TV again, the main thing that stuck with me was the end,
the hole castle part really touched me, its easy to watch, has a great story,
great music, the list goes on and on, its OK me saying how good it is but
everyone will take there own best bits away with them once they have seen it,
yes the animation is top notch and beautiful to watch, it does show its age in a
very few parts but that has now become part of it beauty, i am so glad it has
came out on DVD as it is one of my top 10 films of all time. Buy it or rent it
just see it, best viewing is at night alone with drink and food in reach so you
don't have to stop the film.<br/>
<br/>Enjoy
```

Before we begin tokenizing our input text, we will make a copy of our training data with a few important modifications. We can ignore the unsupervised reviews for now and create a separate validation set to monitor our accuracy while training. We do this by splitting 20% of the training text files into a new directory.

Listing 14.10 Splitting validation from the IMDb dataset

```
train_dir = pathlib.Path("imdb_train")
test_dir = pathlib.Path("imdb_test")
val_dir = pathlib.Path("imdb_val")

shutil.copytree(imdb_extract_dir / "test", test_dir)  ← Moves the test
                                                         data unaltered

val_percentage = 0.2
for category in ("neg", "pos"):
    src_dir = imdb_extract_dir / "train" / category
    src_files = os.listdir(src_dir)
    random.Random(1337).shuffle(src_files)
    num_val_samples = int(len(src_files) * val_percentage)

    os.makedirs(val_dir / category)
    for file in src_files[:num_val_samples]:
        shutil.copy(src_dir / file, val_dir / category / file)
    os.makedirs(train_dir / category)
    for file in src_files[num_val_samples:]:
        shutil.copy(src_dir / file, train_dir / category / file)  ← Splits the training
                                                                     data into a train set
                                                                     and a validation set
```

We are now ready to load the data. Remember how, in chapter 8, we used the `image_dataset_from_directory` utility to create a `Dataset` of images and their labels for a directory structure? You can do the exact same thing for text files using the `text_dataset_from_directory` utility. Let's create three `Dataset` objects for training, validation, and testing.

Listing 14.11 Loading the IMDb dataset for use with Keras

```
from keras.utils import text_dataset_from_directory

batch_size = 32
train_ds = text_dataset_from_directory(train_dir, batch_size=batch_size)
val_ds = text_dataset_from_directory(val_dir, batch_size=batch_size)
test_ds = text_dataset_from_directory(test_dir, batch_size=batch_size)
```

Originally we had 25,000 training and testing examples each, and after our validation split, we have 20,000 reviews to train on and 5,000 for validation. Let's try learning something from this data.

14.4 Set models

The simplest approach we can take regarding the ordering of tokens in text is to discard it. We still tokenize our input reviews normally as a sequence of token IDs, but immediately after tokenization, we convert the entire training example to a set—a simple unordered “bag” of tokens that are either present or absent in a movie review.

The idea here is to use these sets to build a very simple model that assigns a weight to every individual word in a review. The presence of the word “**terrible**” would probably (though not always) indicate a bad review, and “**riveting**” might indicate a good review. We can build a small model that can learn these weights—called a bag-of-words model.

For example, let's say you had a simple input sentence and vocabulary:

```
"this movie made me cry"

{"[UNK]": 0, "movie": 1, "film": 2, "made": 3, "laugh": 4, "cry": 5}
```

We would tokenize this tiny review as

```
[0, 1, 3, 0, 5]
```

Discarding order, we can turn this into a set of token IDs:

```
{0, 1, 3, 5}
```

Finally, we could use a multi-hot encoding to transform the set to a fixed-sized vector with the same length as a vocabulary:

```
[1, 1, 0, 1, 0, 1]
```

The 0 in the fifth position here means the word "laugh" is absent in our review, and the 1 in the sixth position means "cry" is present. This simple encoding of our input review can be used directly to train a model.

14.4.1 Training a bag-of-words model

To do this text processing in code, it would be easy enough to extend our `WordTokenizer` from earlier in the chapter. An even easier solution is to use the `TextVectorization` layer built into Keras. The `TextVectorization` handles word and character tokenization and comes with several additional features, including multi-hot encoding of the layer output.

The `TextVectorization` layer, like many preprocessing layers in Keras, has an `adapt()` method to learn a layer state from input data. In the case of `TextVectorization`, `adapt()` will learn a vocabulary for a dataset on the fly by iterating over an input dataset. Let's use it to tokenize and encode our input data. We will build a vocabulary of 20,000 words, a good starting place for text classification problems.

Listing 14.12 Applying a bag-of-words encoding to the IMDb reviews

```
from keras import layers

max_tokens = 20_000
text_vectorization = layers.TextVectorization(
    max_tokens=max_tokens,
    split="whitespace",
    output_mode="multi_hot",
)
train_ds_no_labels = train_ds.map(lambda x, y: x)
text_vectorization.adapt(train_ds_no_labels)

bag_of_words_train_ds = train_ds.map(
    lambda x, y: (text_vectorization(x), y), num_parallel_calls=8
)
bag_of_words_val_ds = val_ds.map(
    lambda x, y: (text_vectorization(x), y), num_parallel_calls=8
)
bag_of_words_test_ds = test_ds.map(
    lambda x, y: (text_vectorization(x), y), num_parallel_calls=8
)
```

← Learns a word-level vocabulary

Let's look a single batch of our preprocessed input data:

```
>>> x, y = next(bag_of_words_train_ds.as_numpy_iterator())
>>> x.shape
(32, 20000)
>>> y.shape
(32, 1)
```

You can see that after preprocessing, each sample in our batch is converted into a vector of 20,000 numbers, each tracking the presence or absence of a vocabulary term.

Next, we can train a very simple linear model. We will save our model-building code as a function so we can use it again later.

Listing 14.13 Building a bag-of-words regression model

```
def build_linear_classifier(max_tokens, name):
    inputs = keras.Input(shape=(max_tokens,))
    outputs = layers.Dense(1, activation="sigmoid")(inputs)
    model = keras.Model(inputs, outputs, name=name)
    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"],
    )
    return model

model = build_linear_classifier(max_tokens, "bag_of_words_classifier")
```

Let's take a look at our model's summary:

```
>>> model.summary()
Model: "bag_of_words_classifier"
```

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 20000)	0
dense (Dense)	(None, 1)	20,001

```
Total params: 20,001 (78.13 KB)
Trainable params: 20,001 (78.13 KB)
Non-trainable params: 0 (0.00 B)
```

This model is dead simple. We have only 20,001 parameters, one for each word in our vocabulary and one for a bias term. Let's train it. We'll add on the `EarlyStopping` callback first covered in chapter 7, which will automatically stop when training when the validation loss stops improving and restore weights from the best epoch.

Listing 14.14 Training the bag-of-words regression model

```
early_stopping = keras.callbacks.EarlyStopping(
    monitor="val_loss",
    restore_best_weights=True,
    patience=2,
)
history = model.fit(
```

```

bag_of_words_train_ds,
validation_data=bag_of_words_val_ds,
epochs=10,
callbacks=[early_stopping],
)

```

Our model trains in much less than a minute, which is unsurprising given its size. The tokenization and encoding of our input is actually quite a bit more expensive than updating our model parameters. Let's plot the model accuracy (figure 14.2):

```

import matplotlib.pyplot as plt

accuracy = history.history["accuracy"]
val_accuracy = history.history["val_accuracy"]
epochs = range(1, len(accuracy) + 1)

plt.plot(epochs, accuracy, "r--", label="Training accuracy")
plt.plot(epochs, val_accuracy, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.show()

```

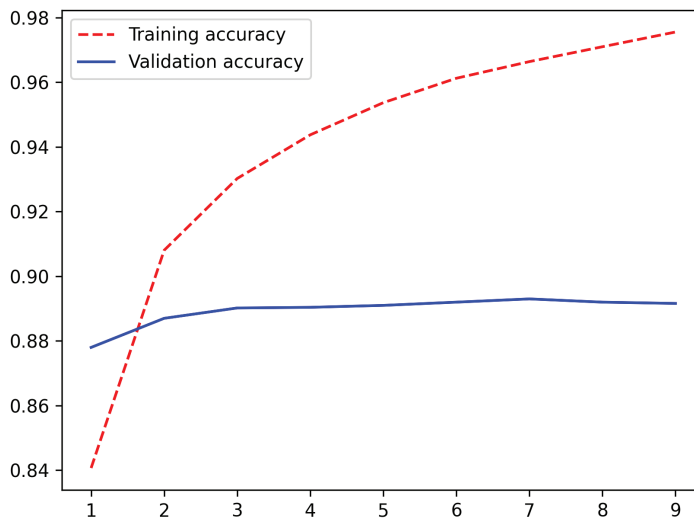


Figure 14 2 Training and validation metrics for our bag of words model

We can see that validation performance levels off rather than significantly declining; our model is so simple it cannot really overfit. Let's try evaluating it on our test set.

Listing 14.15 Evaluating the bag-of-words regression model

```
>>> test_loss, test_acc = model.evaluate(bag_of_words_test_ds)
>>> test_acc
0.88388
```

We can correctly predict the sentiment of a review 88% of the time with a training job light enough that it could run efficiently on a single CPU.

It is worth noting our choice of word tokenization in this example. The reason to avoid character-level tokenization here is pretty obvious—a “bag” of all characters in a movie review will tell you very little about its content. Subword tokenization with a large enough vocabulary would be a good choice, but there is little need for it here. Since the model we are training is so small, it’s convenient to use a vocabulary that is quick to train and have our weights correspond to actual English words.

Preprocessing text efficiently

In all applied machine learning, the speed and efficiency of preprocessing are important concerns. A faster program is always desirable, but this becomes more urgent when the cost of accelerators (GPUs and TPUs) is so high. You want to avoid letting expensive GPUs idle while you preprocess your input!

Text preprocessing is unique because it must always run on a CPU. GPUs strictly handle numeric inputs, so all tokenization must happen before your GPU’s train step. One option is to precompute your tokenized input—tokenization does not depend on model weights, so you could tokenize all input text files and resave them as integer sequences before starting training. However, this is not always practical. Tokenizing text on the fly allows for more rapid experimentation. If you are running inference on an unseen example, there is no way to precompute the tokenized input; you need to tokenize and run a forward pass in rapid succession.

The name of the game when preprocessing text input on the fly is to be “fast enough.” You want to ensure your expensive GPUs always have a new batch of preprocessed data to ingest. If you do that, the GPU is the bottleneck, and there is nothing to gain by improving your tokenization speed.

We’ve seen `tf.data` in previous chapters, and an important reason we use it is that the library is designed to avoid the CPU becoming a bottleneck for a GPU or TPU. We use it throughout this chapter—`keras.utils.text_dataset_from_directory()` will load a `tf.data.Dataset`, and `map()` will transform our input data, for example, by applying a `TextVectorization` layer. `tf.data` works by running text preprocessing in parallel multiple CPU cores, which is generally sufficient to avoid bottlenecking accelerators during a training run.

It is important to note that the code in this chapter is still multi-backend (in fact, we generated the outputs for this chapter using Jax). You can use `tf.data` with PyTorch, JAX, or TensorFlow itself—Keras will automatically convert input Tensors to the correct format for a given backend.

14.4.2 Training a bigram model

Of course, we can intuitively guess that discarding all word order is very reductive because even atomic concepts can be expressed via multiple words: the term “United States” conveys a concept that is quite distinct from the meaning of the words “states” and “united” taken separately. A movie that is “not bad” and a movie that is “bad” should probably get different sentiment scores.

Therefore, it is usually a good idea to inject some knowledge of local word ordering into a model, even for these simple set-based models we are currently building. One easy way to do that is to consider *bigrams*—a term for two tokens that appear consecutively in the input text. Given our example “this movie made me cry,” `{"this", "movie", "made", "me", "cry"}` is the set of all word *unigrams* in the input, and `{"this movie", "movie made", "made me", "me cry"}` is the set of all bigrams. The bag-of-words model we just trained could equivalently be called a unigram model, and the term *n-gram* refers to an ordered sequence of *n* tokens for any *n*.

To add bigrams to our model, we want to consider the frequency of all bigrams while building our vocabulary. We could do this in two ways: by creating a vocabulary of only bigrams or by allowing both bigrams and unigrams to compete for space in the same vocabulary. For the latter case, the term “United States” will be included in our vocabulary before “ventriloquism” if it occurs more frequently in the input text.

Again, we could build this by extending our `WordTokenizer` from earlier in the chapter, but there is no need. `TextVectorization` provides this out of the box. We will train a slightly larger vocabulary to account for the presence of bigrams, `adapt()` a new vocabulary, and multi-hot encode output vectors including bigrams.

Listing 14.16 Applying a bigram encoding to the IMDb reviews

```
max_tokens = 30_000
text_vectorization = layers.TextVectorization(
    max_tokens=max_tokens,
    split="whitespace",
    output_mode="multi-hot",
    ngrams=2,
)
text_vectorization.adapt(train_ds_no_labels)

bigram_train_ds = train_ds.map(
    lambda x, y: (text_vectorization(x), y), num_parallel_calls=8
)
bigram_val_ds = val_ds.map(
    lambda x, y: (text_vectorization(x), y), num_parallel_calls=8
)
bigram_test_ds = test_ds.map(
    lambda x, y: (text_vectorization(x), y), num_parallel_calls=8
)
```

← **Learns a word-level vocabulary**

← **Considers all unigrams and bigrams**

Let’s examine a batch of our preprocessed input again:

```
>>> x, y = next(bigram_train_ds.as_numpy_iterator())
>>> x.shape
(32, 30000)
```

If we look at a small subsection of our vocabulary, we can see both unigram and bigram terms:

```
>>> text_vectorization.get_vocabulary()[100:108]
["in a", "most", "him", "dont", "it was", "one of", "for the", "made"]
```

With our new encoding for our input data, we can train a linear model unaltered from before.

Listing 14.17 Training the bigram regression model

```
model = build_linear_classifier(max_tokens, "bigram_classifier")
model.fit(
    bigram_train_ds,
    validation_data=bigram_val_ds,
    epochs=10,
    callbacks=[early_stopping],
)
```

This model will be slightly larger than our bag-of words models (30,001 parameters instead of 20,001 parameters), but it still trains in about the same amount of time. How did it do?

Listing 14.18 Evaluating the bigram regression model

```
>>> test_loss, test_acc = model.evaluate(bigram_test_ds)
>>> test_acc
0.90116
```

We're now getting 90% test accuracy, a noticeable improvement!

We could improve this number even further by considering trigrams (triplets of words), although beyond trigrams, the problem quickly becomes intractable. The space of possible 4-grams of words in the English language is immense, and the problem grows exponentially as sequences get longer and longer. You would need an immense vocabulary to provide decent coverage of 4-grams, and your model would lose its ability to generalize, simply memorizing entire snippets of sentences with weights attached. To robustly consider longer-ordered text sequences, we will need more advanced modeling techniques.

14.5 Sequence models

Our last two models indicated that sequence information is important. We improved a basic linear model by adding features with some info on local word order.

However, this was done by manually engineering input features, and we can see how the approach will only scale up to a local ordering of just a few words. As is often the case in deep learning, rather than attempting to build these features ourselves, we should expose the model to the raw word sequence and let it directly learn positional dependencies between tokens.

Models that ingest a complete token sequence are called, simply enough, *sequence models*. We have a few choices for architecture here. We could build an RNN model as we just did for timeseries modeling. We could build a 1D ConvNet, similar to our image processing models, but convolving filters over a single sequence dimension. And as we will dig into in the next chapter, we can build a Transformer.

Before taking on any of these approaches, we must preprocess our inputs into ordered sequences. We want an integer sequence of token IDs, as we saw in the tokenization portion of this chapter, but with one additional wrinkle to handle. When we run computations on a batch of inputs, we want all inputs to be rectangular so all calculations can be effectively parallelized across the batch on a GPU. However, tokenized inputs will almost always have varying lengths. IMDb movie reviews range from just a few sentences to multiple paragraphs, with varying word counts.

To accommodate this fact, we can truncate our input sequences or “pad” them with another special token “[PAD]”, similar to the “[UNK]” token we used earlier. For example, given two input sentences and a desired length of eight

```
"the quick brown fox jumped over the lazy dog"  
"the slow brown badger"
```

we would tokenize to the integer IDs for the following tokens:

```
["the", "quick", "brown", "fox", "jumped", "over", "the", "lazy"]  
["the", "slow", "brown", "badger", "[PAD]", "[PAD]", "[PAD]", "[PAD]"]
```

This will allow our batch computation to proceed much faster, although we will need to be careful with our padding tokens to ensure they do not affect the quality of our model predictions.

To keep a manageable input size, we can truncate our IMDb reviews after the first 600 words. This is a reasonable choice, since the average review length is 233 words, and only 5% of reviews are longer than 600 words. Once again, we can use the `Vectorization` layer, which has an option for padding or truncating inputs and includes a “[PAD]” at index zero of the learned vocabulary.

Listing 14.19 Padding IMDb reviews to a fixed sequence length

```

max_length = 600
max_tokens = 30_000
text_vectorization = layers.TextVectorization(
    max_tokens=max_tokens,
    split="whitespace",
    output_mode="int",
    output_sequence_length=max_length,
)
text_vectorization.adapt(train_ds_no_labels)

sequence_train_ds = train_ds.map(
    lambda x, y: (text_vectorization(x), y), num_parallel_calls=8
)
sequence_val_ds = val_ds.map(
    lambda x, y: (text_vectorization(x), y), num_parallel_calls=8
)
sequence_test_ds = test_ds.map(
    lambda x, y: (text_vectorization(x), y), num_parallel_calls=8
)

```

Learns a word-level vocabulary

Outputs a integer sequence of token IDs

Pads and truncates to 600 tokens

Let's take a look at a single input batch:

```

>>> x, y = next(sequence_test_ds.as_numpy_iterator())
>>> x.shape
(32, 600)
>>> x
array([[ 11,    29,    7, ...,  0,    0,    0],
       [ 132,   115,   35, ...,  0,    0,    0],
       [1825,    3, 25819, ...,  0,    0,    0],
       ...,
       [  4,   576,   56, ...,  0,    0,    0],
       [ 30,   203,    4, ...,  0,    0,    0],
       [5104,    1,   14, ...,  0,    0,    0]])

```

Each batch has the shape `(batch_size, sequence_length)` after preprocessing, and almost all training samples have a number of 0s for padding at the end.

14.5.1 Training a recurrent model

Let's try training an LSTM. As we saw in the previous chapter, LSTMs can work efficiently with sequence data. Before we can apply it, we still need to map our token ID *integers* into floating-point data ingestible by a `Dense` layer.

The most straightforward approach is to *one-hot* our input IDs, similar to the multi-hot encoding we did for an entire sequence. Each token will become a long vector with all 0s and a single 1 at the index of the token in our vocabulary. Let's build a layer to one-hot encode our input sequence.

Listing 14.20 Building a one-hot encoding layer with Keras ops

```

from keras import ops

class OneHotEncoding(keras.Layer):
    def __init__(self, depth, **kwargs):
        super().__init__(**kwargs)
        self.depth = depth

    def call(self, inputs):
        flat_inputs = ops.reshape(ops.cast(inputs, "int"), [-1])
        one_hot_vectors = ops.eye(self.depth)
        outputs = ops.take(one_hot_vectors, flat_inputs, axis=0)
        return ops.reshape(outputs, ops.shape(inputs) + (self.depth,))

one_hot_encoding = OneHotEncoding(max_tokens)

```

Uses our input token IDs to gather the correct vector for each token

Builds an identity matrix with all possible one-hot vectors

Flattens the inputs

Unflattens the output

Let's try this layer out on a single input batch:

```

>>> x, y = next(sequence_train_ds.as_numpy_iterator())
>>> one_hot_encoding(x).shape
(32, 600, 30000)

```

We can build this layer directly into a model and use a bidirectional LSTM to allow information to propagate both forward and backward along the token sequence. Later, when we look at generation, we will see the need for unidirectional sequence models (where a token state only depends on the token state before it). For classification tasks, a bidirectional LSTM is a good fit.

Let's build our model.

Listing 14.21 Building an LSTM sequence model

```

hidden_dim = 64
inputs = keras.Input(shape=(max_length,), dtype="int32")
x = one_hot_encoding(inputs)
x = layers.Bidirectional(layers.LSTM(hidden_dim))(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs, name="lstm_with_one_hot")
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"],
)

```

We can take a look at our model summary to get a sense of our parameter count:

```
>>> model.summary()
Model: "lstm_with_one_hot"
```

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 600)	0
one_hot_encoding (OneHotEncoding)	(None, 600, 30000)	0
bidirectional (Bidirectional)	(None, 128)	15,393,280
dropout (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 1)	129

```
Total params: 15,393,409 (58.72 MB)
Trainable params: 15,393,409 (58.72 MB)
Non-trainable params: 0 (0.00 B)
```

This is quite the step up in size from the unigram and bigram models. At about 15 million parameters, this is one of the larger models we have trained in the book so far, with only a single LSTM layer. Let's try training the model.

Listing 14.22 Training the LSTM sequence model

```
model.fit(
    sequence_train_ds,
    validation_data=sequence_val_ds,
    epochs=10,
    callbacks=[early_stopping],
)
```

How does it perform?

Listing 14.23 Evaluating the LSTM sequence model

```
>>> test_loss, test_acc = model.evaluate(sequence_test_ds)
>>> test_acc
0.84811
```

This model works, but it trains very slowly, especially compared to the lightweight model of the previous section. That's because our inputs are quite large: each input sample is encoded as a matrix of size (600, 30000) (600 words per sample, 30,000 possible words). That is 18,000,000 floating-point numbers for a single movie review! Our bidirectional LSTM has a lot of work to do. In addition to being slow, the model only gets to 84% test accuracy—it doesn't perform nearly as well as our very fast set-based models.

Clearly, using one-hot encoding to turn words into vectors, which was the simplest thing we could do, wasn't a great idea. There's a better way—word embeddings.

14.5.2 Understanding word embeddings

When you encode something via one-hot encoding, you're making a feature engineering decision. You're injecting into your model a fundamental assumption about the structure of your feature space. That assumption is that the different tokens you're encoding are all independent from each other: indeed, one-hot vectors are all orthogonal to one another. In the case of words, that assumption is clearly wrong. Words form a structured space: they share information with each other. The words “movie” and “film” are interchangeable in most sentences, so the vector that represents “movie” should not be orthogonal to the vector that represents “film”—they should be the same vector, or close enough.

To get more abstract, the geometric relationship between two-word vectors should reflect the semantic relationship between these words. For instance, in a reasonable word vector space, you would expect synonyms to be embedded into similar word vectors, and in general, you would expect the geometric distance (such as the cosine distance or L2 distance) between any two-word vectors to relate to the “semantic distance” between the associated words. Words that mean different things should lie far away from each other, whereas related words should be closer.

Word embeddings are vector representations of words that achieve precisely this: they map human language into a structured geometric space.

Whereas the vectors obtained through one-hot encoding are binary, sparse (mostly made of zeros), and very high-dimensional (the same dimensionality as the number of words in the vocabulary), word embeddings are low-dimensional floating-point vectors (that is, dense vectors, as opposed to sparse vectors); see figure 14.3. It's common to see word embeddings that are 256-dimensional, 512-dimensional, or 1,024-dimensional when dealing with very large vocabularies. On the other hand, one-hot encoding words generally leads to vectors that are 30,000-dimensional in the case of our current vocabulary. So word embeddings pack more information into far fewer dimensions.

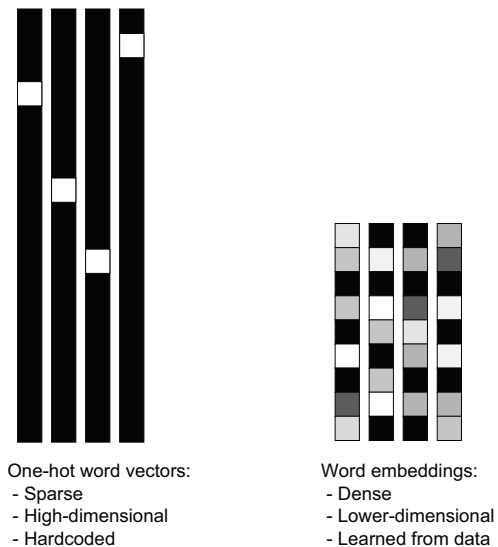


Figure 14.3 Word representations obtained from one-hot encoding or hashing are sparse, high-dimensional, and hardcoded. Word embeddings are dense, relatively low-dimensional, and learned from data.

Besides being dense representations, word embeddings are also structured representations, and their structure is learned from data. Similar words get embedded in close locations, and further, specific directions in the embedding space are meaningful. To make this clearer, let's look at a concrete example. In figure 14.4, four words are embedded on a 2D plane: cat, dog, wolf, and tiger. With the vector representations we chose here, some semantic relationships between these words can be encoded as geometric transformations. For instance, the same vector allows us to go from cat to tiger and from dog to wolf: this vector could be interpreted as the “from pet to wild animal” vector. Similarly, another vector lets us go from dog to cat and from wolf to tiger, which could be interpreted as a “from canine to feline” vector.

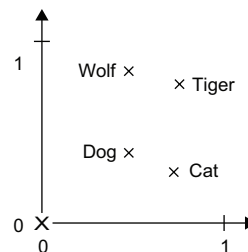


Figure 14.4 A toy example of a word-embedding space

In real-world word-embedding spaces, typical examples of meaningful geometric transformations are “gender” vectors and “plural” vectors. For instance, by adding a “female” vector to the vector “king,” we obtain the vector “queen.” By adding a “plural” vector, we obtain “kings.” Word-embedding spaces typically feature thousands of such interpretable and potentially useful vectors.

Let's look at how to use such an embedding space in practice.

14.5.3 Using a word embedding

Is there an ideal word-embedding space that perfectly maps human language and can be used for any NPL task? Possibly, but we have yet to compute anything of the sort. Also, there is no single human language we could attempt to map—there are many different languages, and they aren't isomorphic to one another because a language is the reflection of a specific culture and a particular context. More pragmatically, what makes a good word-embedding space depends heavily on your task: the perfect word-embedding space for an English-language movie review sentiment-analysis model may look different from the ideal embedding space for an English-language legal document classification model because the importance of certain semantic relationships varies from task to task.

It's thus reasonable to learn a new embedding space with every new task. Fortunately, backpropagation makes this easy, and Keras makes it even easier. It's about learning the weights of the Keras `Embedding` layer.

The `Embedding` layer is best understood as a dictionary that maps integer indices (which stand for specific words) to dense vectors. It takes integers as input, looks them up in an internal dictionary, and returns the associated vectors. It's effectively a dictionary lookup (see figure 14.5).

The `Embedding` layer takes as input a rank-2 tensor with shape `(batch_size, sequence_length)`, where each entry is a sequence of integers. The layer returns a floating-point tensor of shape `(batch_size, sequence_length, embedding_size)`.

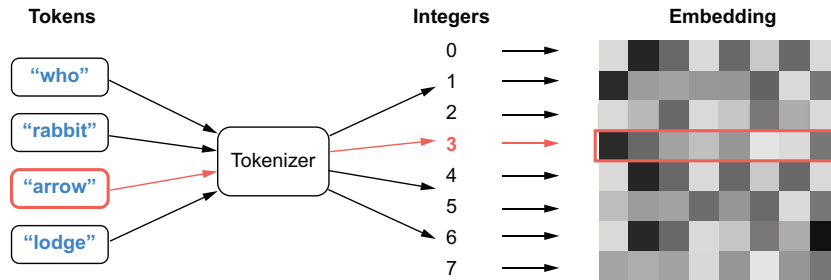


Figure 14.5 An **Embedding** layer acts as a dictionary mapping ints to floating point vectors.

When you instantiate an **Embedding** layer, its weights (its internal dictionary of token vectors) are initially random, just as with any other layer. During training, these word vectors are gradually adjusted via backpropagation, structuring the space into something the downstream model can exploit. Once fully trained, the embedding space will show a lot of structure—a kind of structure specialized for the specific problem for which you’re training your model.

Let’s build a model with an **Embedding** layer and benchmark it on our task.

Listing 14.24 Building an LSTM sequence model with an **Embedding** layer

```
hidden_dim = 64
inputs = keras.Input(shape=(max_length,), dtype="int32")
x = keras.layers.Embedding(
    input_dim=max_tokens,
    output_dim=hidden_dim,
    mask_zero=True,
)(inputs)
x = keras.layers.Bidirectional(keras.layers.LSTM(hidden_dim))(x)
x = keras.layers.Dropout(0.5)(x)
outputs = keras.layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs, name="lstm_with_embedding")
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"],
)
```

The first two arguments to the **Embedding** layer are fairly straightforward. `input_dim` sets the total range of possible values for the integer inputs to the layer—that is, how many possible keys are there in our dictionary lookup. `output_dim` sets the dimensionality of the output vector we look up—that is, the dimensionality of our structured vector space for words.

The third argument, `mask_zero=True`, is a little more subtle. This argument tells Keras which inputs in our sequence are "[PAD]" tokens, so we can *mask* these entries later in the model.

Remember that when preprocessing our sequence input, we might add a lot of padding tokens to our original input so that a token sequence might look like:

```
["the", "movie", "was", "awful", "[PAD]", "[PAD]", "[PAD]", "[PAD]"]
```

All of those padding tokens will be first embedded and then fed into the LSTM layer. This means the last representation we receive from the LSTM cell might contain the results of processing the "[PAD]" token representation over and over recurrently. We are not very interested in the learned LSTM representation for the last "[PAD]" token in the previous sequence. Instead, we are interested in the representation of "awful", the last non-padding token. Or, put equivalently, we want to mask all of the "[PAD]" tokens so that they do not affect our final output prediction.

`mask_zero=True` is simply a shorthand to easily do such masking in Keras with the Embedding layer. Keras will mark all elements in our sequence that initially contained a zero value, where zero is assumed to be the token ID for the "[PAD]" token. This mask will be used internally by the LSTM layer. Instead of outputting the last learned representation for the whole sequence, it will output the last non-masked representation.

This form of masking is *implicit* and easy to use, but you can always be explicit about which items in a sequence you would like to mask if the need arises. The LSTM layer takes an optional `mask` call argument, for explicit or custom masking.

Before we train this new model, let's take a look at the model summary:

```
>>> model.summary()
Model: "lstm_with_embedding"
```

Layer (type)	Output Shape	Param #	Connected to
input_layer_3 (InputLayer)	(None, 600)	0	-
embedding (Embedding)	(None, 600, 64)	1,920,000	input_layer_3[0][...]
not_equal (NotEqual)	(None, 600)	0	input_layer_3[0][...]
bidirectional_1 (Bidirectional)	(None, 128)	66,048	embedding[0][0], not_equal[0][0]
dropout_1 (Dropout)	(None, 128)	0	bidirectional_1[0][...]
dense_3 (Dense)	(None, 1)	129	dropout_1[0][0]


```
Total params: 1,986,177 (7.58 MB)
Trainable params: 1,986,177 (7.58 MB)
Non-trainable params: 0 (0.00 B)
```

We have reduced the number of parameters for our one-hot-encoded LSTM model from 15 million to 2 million. Let's train and evaluate the model.

Listing 14.25 Training and evaluating the LSTM with an Embedding layer

```
>>> model.fit(
...     sequence_train_ds,
...     validation_data=sequence_val_ds,
...     epochs=10,
...     callbacks=[early_stopping],
... )
>>> test_loss, test_acc = model.evaluate(sequence_test_ds)
>>> test_acc
0.8443599939346313
```

With the embedding, we have reduced both our training time and model size by an order of magnitude. A learned embedding is clearly far more efficient than one-hot encoding our input.

However, the LSTM's overall performance did not change. Accuracy was stubbornly around 84%, still a far cry from the bag-of-words and bigram models. Does this mean that a “structured embedding space” for input tokens is not that practically useful? Or is it not useful for text classification tasks?

Quite the contrary, a well-trained token embedding space can dramatically improve the practical performance ceiling of a model like this. The issue in this particular case is with our training setup. We lack enough data in our 20,000 review examples to effectively train a good word embedding. By the end of our 10 training epochs, our train set accuracy has cracked 99%. Our model has begun to overfit and memorize our input, and it turns out it is doing so well before we have learned an optimal set of word embeddings for the task at hand.

For cases like this, we can turn to *pretraining*. Rather than training our word embedding jointly with the classification task, we can train it separately, on more data, without the need for positive and negative review labels. Let's take a look.

Augmenting text data

After seeing the importance of data augmentation for computer vision problems, you might wonder if we can do the same for text. The short answer is yes, you can, although it is not nearly as effective in the text domain.

Basic text augmentation techniques look for basic edits we can make to input text that might help make our model more robust. For example, we might randomly delete

(continued)

or swap the position of words in a sentence, so the sentence “The rain in Spain falls mainly on the plain” becomes “The rain Spain falls plain on the mainly.” Training your model on such edited inputs can make it robust to typos and grammar mistakes.

However, this example also concisely shows the big pitfall with text augmentation—it is easy to alter the meaning of an input example unintentionally. Unlike image data, where you can crop, rotate, and adjust color levels on a picture of a cat and still have a recognizable cat on the other end, language is order-dependent and highly sensitive to tiny changes. A sentence with two words swapped might mean the opposite of the input sentence. Some augmentation techniques seek to solve this by replacing words from a table of known synonyms, but this, too, can be brittle if we choose the incorrect meaning of a word. These issues have kept text augmentation from being widespread in practice. It is usually a better idea to seek more text examples than to sink time into text augmentation techniques.

Generative models, which we will see in the coming chapters, are beginning to offer a new form of text augmentation that can alleviate these pain points. By generating outputs from a model that has learned how to produce consistent and coherent text, we can create completely unseen inputs that plausibly resemble our input data. This poses its own challenges but opens up a new frontier for text augmentation on problems where data is particularly sparse and challenging to collect.

14.5.4 Pretraining a word embedding

The last decade of rapid advancement in NLP has coincided with the rise of *pretraining* as the dominant approach for text modeling problems. Once we move past simple set-based regression models to sequence models with millions or even billions of parameters, text models become incredibly data-hungry. We are usually limited by our ability to find labeled examples for a particular problem in the text domain.

The idea is to devise an unsupervised task to train model parameters that do not need labeled data. Pretraining data can be text in a similar domain to our final task, or even just arbitrary text in the languages we are interested in working with. Pretraining allows us to learn general patterns in language, effectively priming our model before we specialize it to the final task we are interested in.

Word embeddings were one of the first big successes with text pretraining, and we will show how to pretrain a word embedding in this section. Remember the `unsup/` directory we ignored in our IMDb dataset preparation? It contains another 25,000 reviews—the same size as our training data. We will combine all our training data together and show how to pretrain the parameters of an `Embedding` layer with an unsupervised task.

One of the most straightforward setups for training a word embedding is called the Continuous Bag of Words (CBOW) model.² The idea is to slide a window over all the

² Mikolov et al., “Efficient Estimation of Word Representations in Vector Space,” International Conference on Learning Representations (2013), <https://arxiv.org/abs/1301.3781>.

text in a dataset, where we continuously attempt to guess a missing word based on the words that appear to its direct right and left (figure 14.6). For example, if our “bag” of surrounding words contained the words “sail,” “wave,” and “mast,” we might guess that the middle word is “boat” or “ocean.”

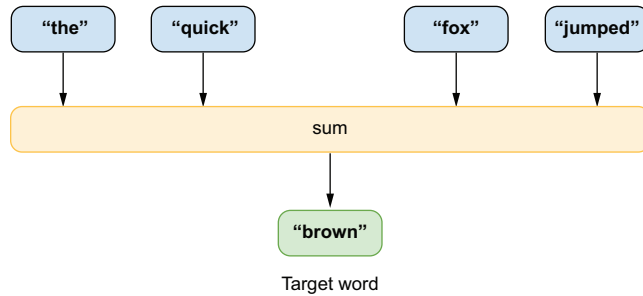


Figure 14.6 The Continuous Bag of Words predicts a word based on its surrounding context with a shallow neural network.

In our particular IMDb classification problem, we are interested in “priming” the word embedding of the LSTM model we just trained. We can reuse the `TextVectorization` vocabulary we computed earlier. All we are trying to do here is to learn a good 64-dimensional vector for each word in this vocabulary.

We can create a new `TextVectorization` layer with the same vocabulary that does not truncate or pad input. We will preprocess the output tokens of this layer by sliding a context window across our text.

Listing 14.26 Removing padding from our `TextVectorization` preprocessing layer

```
imdb_vocabulary = text_vectorization.get_vocabulary()
tokenize_no_padding = keras.layers.TextVectorization(
    vocabulary=imdb_vocabulary,
    split="whitespace",
    output_mode="int",
)
```

To preprocess our data, we will slide a window across our training data, creating “bags” of nine consecutive tokens. Then, we use the middle word as our label and the remaining eight words as an unordered context to predict our label.

To do this, we will again use `tf.data` to preprocess our inputs, although this choice does not limit the backend we use for actual model training.

Listing 14.27 Preprocessing our IMDb data for pretraining a CBOW model

```
import tensorflow as tf

context_size = 4
```

Words to the left
or right of label

```

window_size = 9  ← Total window size

def window_data(token_ids):
    num_windows = tf.maximum(tf.size(token_ids) - context_size * 2, 0)
    windows = tf.range(window_size)[None, :]
    windows = windows + tf.range(num_windows)[:, None]
    windowed_tokens = tf.gather(token_ids, windows)
    return tf.data.Dataset.from_tensor_slices(windowed_tokens)

def split_label(window):
    left = window[:context_size]
    right = window[context_size + 1 :]
    bag = tf.concat((left, right), axis=0)
    label = window[4]
    return bag, label

dataset = keras.utils.text_dataset_from_directory(
    imdb_extract_dir / "train", batch_size=None
)
dataset = dataset.map(lambda x, y: x, num_parallel_calls=8)
dataset = dataset.map(tokenize_no_padding, num_parallel_calls=8)
dataset = dataset.interleave(window_data, cycle_length=8, num_parallel_calls=8)
dataset = dataset.map(split_label, num_parallel_calls=8)

```

Tokenizes

Drops label

Uses all training data, including the unsup/ directory

Splits middle wonder into a label

Creates context windows

After preprocessing, we can see that we have eight integer token IDs as context paired with a single token ID label.

The model we train with this data is exceedingly simple. We will use an `Embedding` layer to embed all context tokens and a `GlobalAveragePooling1D` to compute the average embedding of our “bag” of context tokens. Then, we use that average embedding to predict the value of our middle label token.

That’s it! By repeatedly refining our embedding space so that we are good at predicting a word based on nearby word embeddings, we learn a rich embedding of tokens used in movie reviews.

Listing 14.28 Building a CBOW model

```

hidden_dim = 64
inputs = keras.Input(shape=(2 * context_size,))
cbow_embedding = layers.Embedding(
    max_tokens,
    hidden_dim,
)
x = cbow_embedding(inputs)
x = layers.GlobalAveragePooling1D()(x)

```

```

outputs = layers.Dense(max_tokens, activation="sigmoid")(x)
cbow_model = keras.Model(inputs, outputs)
cbow_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["sparse_categorical_accuracy"],
)
>>> cbow_model.summary()
Model: "functional_1"

```

Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 8)	0
embedding_1 (Embedding)	(None, 8, 64)	1,920,000
global_average_pooling1d_2 (GlobalAveragePooling1D)	(None, 64)	0
dense_4 (Dense)	(None, 30000)	1,950,000

```

Total params: 3,870,000 (14.76 MB)
Trainable params: 3,870,000 (14.76 MB)
Non-trainable params: 0 (0.00 B)

```

Because our model is so simple, we can use a large batch size to speed up training without worrying about memory constraints.

We will also call `cache()` on this batched dataset so that we store the entire preprocessed dataset in memory rather than recomputing it each epoch. This is because for this very simple model, we are bottlenecked on preprocessing rather than training. That is, it is slower to tokenize our text and compute sliding windows on the CPU than to update our model parameters on the GPU.

In such cases, saving your preprocessed outputs in memory or on disk is usually a good idea. You will notice how our later epochs are more than three times faster than the first. This is thanks to the cache of preprocessed training data.

Listing 14.29 Training the CBOW model

```

dataset = dataset.batch(1024).cache()
cbow_model.fit(dataset, epochs=4)

```

At the end of training, we are able to guess the middle word around 12% of the time based solely on the neighboring eight words. This may not sound like a great result, but given that we have 30,000 words to guess from each time, this is actually a reasonable accuracy score.

Let's use this word embedding to improve the performance of our LSTM model.

14.5.5 Using the pretrained embedding for classification

Now that we have trained a new word embedding, applying it to our LSTM model is simple. First, we create the model precisely as we did before.

Listing 14.30 Building another LSTM sequence model with an Embedding layer

```
inputs = keras.Input(shape=(max_length,))
lstm_embedding = layers.Embedding(
    input_dim=max_tokens,
    output_dim=hidden_dim,
    mask_zero=True,
)
x = lstm_embedding(inputs)
x = layers.Bidirectional(layers.LSTM(hidden_dim))(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs, name="lstm_with_cbow")
```

Then, we apply our embedding weights from the CBOW embedding layer to the LSTM embedding layer. This effectively acts as a new and better initializer for the roughly 2 million embedding parameters in the LSTM model.

Listing 14.31 Reusing the CBOW embedding to prime the LSTM model

```
lstm_embedding.embeddings.assign(cbow_embedding.embeddings)
```

With that, we can compile and train our LSTM model as normal.

Listing 14.32 Training the LSTM model with a pretrained embedding.

```
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"],
)
model.fit(
    sequence_train_ds,
    validation_data=sequence_val_ds,
    epochs=10,
    callbacks=[early_stopping],
)
```

Let's evaluate our LSTM model.

Listing 14.33 Evaluating the LSTM model with a pretrained embedding

```
>>> test_loss, test_acc = model.evaluate(sequence_test_ds)
>>> test_acc
0.89139
```

With the pretrained embedding weights, we have boosted our LSTM performance to roughly the same as our set-based models. We do slightly better than the unigram model and slightly worse than the bigram model.

This may seem like a bit of a letdown after all the work we put in. Is learning on the entire sequence, with order information, just a bad idea? The problem is that we are *still data-constrained* for our final LSTM model. The model is expressive and powerful enough that with enough movie reviews, we would easily outperform set-based approaches, but we need a lot more training on *ordered data* before our model's performance ceiling is reached.

This is an easily solvable problem with enough compute resources. In the next chapter, we will cover the transformer model. The model is slightly better at learning dependencies across longer token sequences, but most critically, these models are often trained on large amounts of English text, including all word order information. This allows the model to learn, roughly speaking, a statistical form of the grammatical patterns that govern language. These types of statistical patterns around word order are precisely why our current LSTM model is too data-constrained to learn effectively.

However, as we move on to large, more advanced models that will push the limits of text-classification performance, it is worth pointing out that simple set-based regression approaches like our bigram model give you a lot of “bang for your buck.” Set-based models are lightning-fast and can contain just a few thousand parameters, a far cry from the billion-parameter large language models that dominate the news today.

If you are working in an environment where compute is limited and you can sacrifice some accuracy, set-based models can often be the most cost-effective approach.

Summary

- All text modeling problems involve a preprocessing step where text is broken up and transformed to integer data, called *tokenization*.
- Tokenization can be divided into three steps: *standardization*, *splitting*, and *indexing*. Standardization normalizes text, splitting breaks text up into tokens, and indexing assigns each token a unique integer ID.
- There are three main types of tokenization: *character*, *word*, and *subword* tokenization. With an expressive-enough model and sufficient training data, the *subword* tokenization is usually the most effective.
- NLP models differ primarily in handling the order of input tokens:
 - *Set models* discard most order information and learn simple and fast models based solely on the presence or absence of tokens in the input. *Bigram* or *trigram* models consider the presence or absence of two or three consecutive tokens. Set models are incredibly fast to train and deploy.
 - *Sequence models* attempt to learn with the ordered sequence of tokens in the input data. Sequence models need large amounts of data to learn effectively.

- An *embedding* is an efficient way to transform token IDs into a learned latent space. Embeddings can be trained normally with gradient descent.
- *Pretraining* is critical for sequence models as a way to get around the data-hungry nature of these models. During *pretraining*, an unsupervised task allows models to learn from large amounts of unlabeled text data. The learned parameters can then be transferred to a downstream task.